UNIT - IV

THE MEMORY ORGANIZATION

Computer Organization - Handwritten Study Notes

Syllabus topics:

Basic Concepts, Semiconductor RAM Memories, Read-Only Memories, Speed-Size-Cost, Cache Memories, Performance Considerations, Virtual Memories, Memory Management Requirements, Secondary Storage

I. Basic Concepts

Memory is the place where **programs and data are stored** . The processor fetches instructions and data from memory and writes the results back into it.

- Memory is made of millions of **storage cells** ; each cell stores **1 bit** (0 or 1).
- Bits are grouped into **words** (word length = 8, 16, 32 or 64 bits). A word is the unit of transfer.
- Every word has a unique **address** . A k-bit address can reach 2^k locations: 16-bit address -> 2^16 = 64K words, 32-bit address -> 2^32 = 4G.
- Most machines are **byte-addressable** (each byte has its own address).

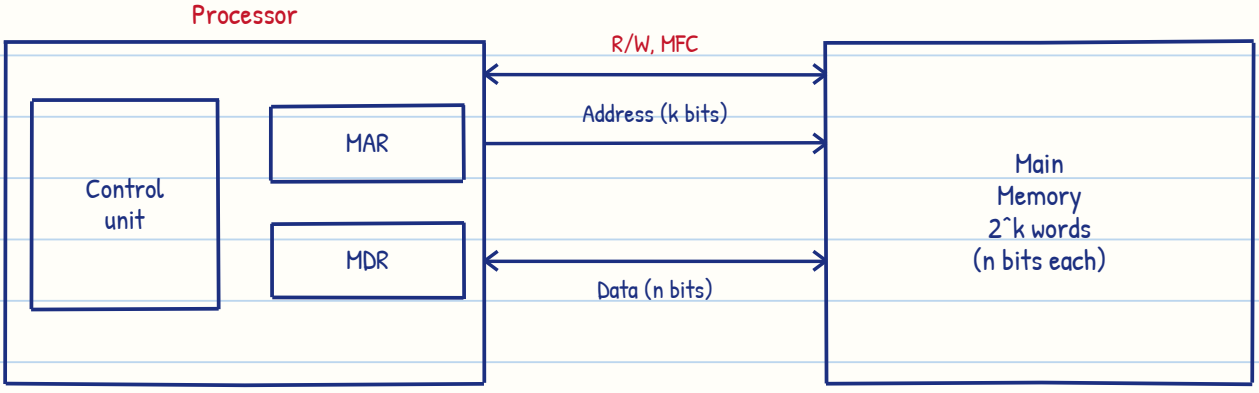


Fig 1: Connection of the processor and the memory

Important registers and signals

- **MAR** (Memory Address Register) holds the address of the location to be accessed.
- **MDR** (Memory Data Register) holds the data being read from / written to memory.
- **R/W** control line tells memory whether to Read or Write. **MFC** (Memory Function

Completed) is the reply signal from memory.

Read operation (step by step)

1. Processor loads the address into **MAR** ; it appears on the address lines.
2. Processor sets **R/W = 1** (Read) and starts the operation.
3. Memory places the word on the data lines and sets **MFC** .
4. Processor copies the data into **MDR** .

- **Write** is the same, but the data is first put in MDR and R/W = 0 (Write); memory stores it and sets MFC.

Key definitions

Memory access time = time between starting an operation (Read signal) and its completion (data available). **Memory cycle time** = minimum delay between two successive memory operations (always >= access time). **RAM** = Random Access Memory: any location is reached in the same time. **Bandwidth** = number of bits/bytes transferred per second.

2. Semiconductor RAM Memories

2.1 Internal organisation of a memory chip

Cells are arranged in a **rows x columns array** . All cells of a row form one word and share a **word line** . Each column has **bit lines** that connect to a **Sense/Write circuit** .

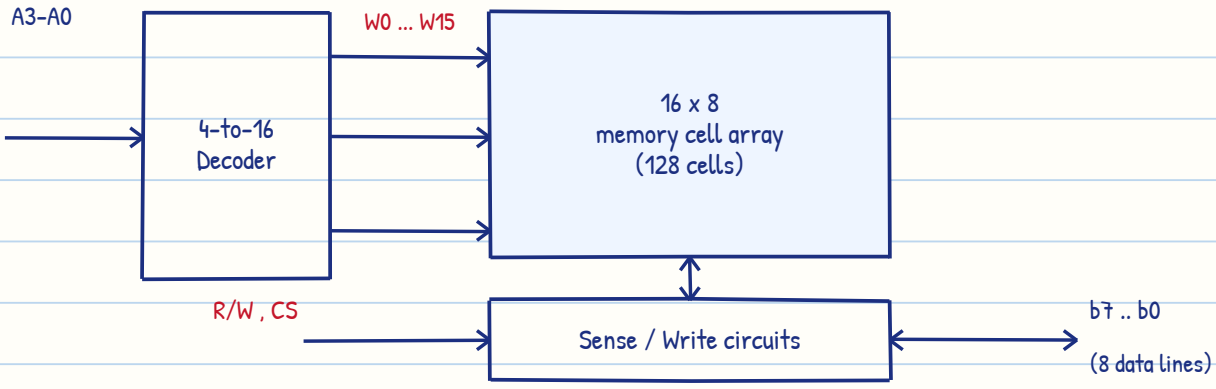


Fig 2: Organisation of a 16 x 8 memory chip

- 16 words x 8 bits = 128 cells. 16 words need **4 address bits** (A3-A0); the **decoder** activates exactly one word line.
- **R/W** selects the operation; **CS** (Chip Select) enables the chip when many chips are used.

- External pins: 4 address + 8 data + R/W + CS + 2 power = **16 pins** .
- Bigger chips save pins by organising the array as a square. Example **1K x 1** chip: 32 x 32 array, **5-bit row decoder** + **5-bit column selector** = 10 address bits.

2.2 Static RAM (SRAM)

SRAM cells hold their state **as long as power is ON** (no refresh needed). Memory that loses data when power is off is called **volatile** .

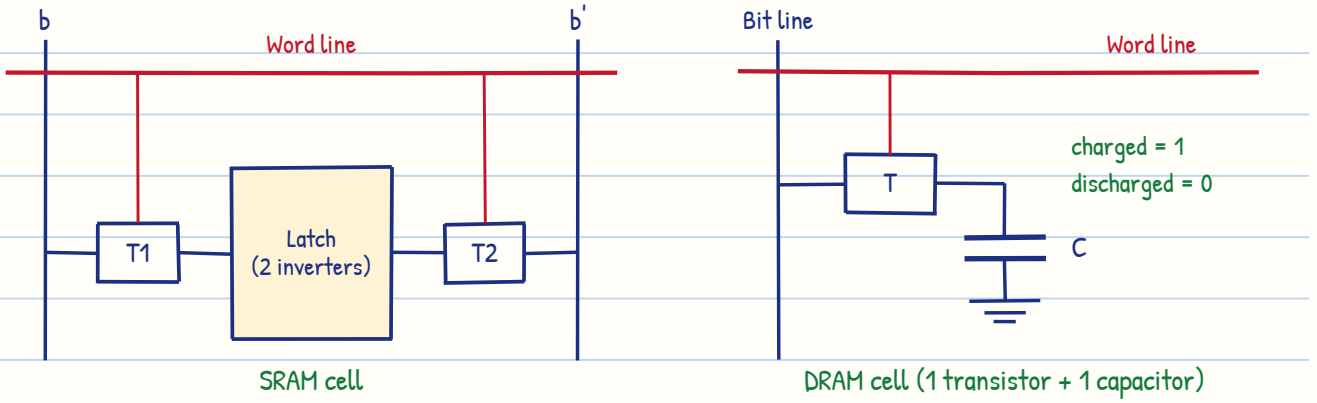


Fig 3: SRAM cell (left) and DRAM cell (right)

- The cell is a **latch** of two cross-coupled inverters. Two transistors **T1, T2** connect it to bit lines **b** and **b'** .
- **Read:** word line = 1 -> T1, T2 ON -> latch value appears on b and b' -> sense circuit reads it.
- **Write:** Sense/Write circuit puts the value on b and its complement on b', then word line = 1 -> latch is forced to the new state.
- CMOS SRAM cell uses **6 transistors** . Very **fast (a few ns)** and low power, but **expensive** and low density. Used for **cache memory** .

2.3 Dynamic RAM (DRAM)

- Cell = **1 transistor + 1 capacitor** . Charged capacitor = 1, discharged = 0. Much smaller and cheaper than SRAM, so used for **main memory** .
- The capacitor **leaks** its charge, so every row must be **refreshed** (re-written) every few milliseconds (typically 16-64 ms).
- **Read:** word line = 1 -> transistor ON -> charge goes to bit line -> sense amplifier detects it and **rewrites** the value (restore).
- **Write:** bit line is set to the required voltage, word line = 1, capacitor is charged or discharged.

Asynchronous DRAM

- Address is sent in two halves on the **same pins** : **row address** with **RAS** (Row Address

Strobe), then **column address** with **CAS** (Column Address Strobe). This saves pins.

- **Fast Page Mode:** once a row is selected, many columns of that row can be read by changing only the column address and pulsing CAS -> fast block transfers.

Synchronous DRAM (SDRAM)

- Operations are **synchronised with a clock** from the processor/bus. Row is read into a latch (row buffer) and consecutive columns come out in **bursts** (e.g. 4 or 8 words) with a single address.
- **DDR SDRAM** (Double Data Rate) transfers data on **both rising and falling clock edges** -> twice the bandwidth. DDR2, DDR3, DDR4... are later generations with higher speed and lower voltage.
- **Latency** = time to get the first word; **Bandwidth** = words per second. Both decide memory performance.

2.4 Building large memories from chips

Example: build a **2M x 32** memory (2M = 2^21 words) using **512K x 8** chips.

- Chips in one row = 32 / 8 = **4** (to make a 32-bit word). Rows = 2M / 512K = **4** . Total chips = 4 x 4 = **16** .
- 21-bit address: the lower **19 bits** (2^19 = 512K) go to every chip; the upper **2 bits** go to a **2-to-4 decoder** that selects one row of chips (Chip Select).
- Dynamic memories also need a **memory controller** : it multiplexes the address into row/column parts, generates RAS/CAS and does the **refresh** automatically.
- DRAM chips are sold as small boards called **SIMMs / DIMMs** (Single/Dual In-line Memory Modules).

Feature	SRAM	DRAM
Cell	6 transistors (latch)	1 transistor + 1 capacitor
Refresh	Not needed	Needed every few ms
Speed	Very fast	Slower
Cost / density	Costly, low density	Cheap, high density
Used for	Cache	Main memory

3. Read-Only Memories (ROM)

ROM is **non-volatile** : contents stay even when power is off. Normal operation only **reads** it. It stores the **bootstrap loader / firmware** that starts the computer.

- **Storing a bit:** a transistor connects the bit line to ground -> bit = 0; no connection -> bit = 1. The word line activates the row and the bit line shows 0 or 1.

Type	How programmed / erased	Remarks
ROM (mask)	Programmed by the manufacturer during fabrication	Cheap only for large volumes; cannot change
PROM	User programs once by blowing fuses (high current)	One-time programmable
EPROM	Erased by ultraviolet light (whole chip), then reprogrammed	Has a transparent quartz window
EEPROM	Erased and written electrically, cell by cell	No need to remove the chip; slow, costly
Flash	Electrically erased in blocks, written in bytes/pages	High density, low power; used in USB drives, memory cards, SSD

Note: Flash memory has a limited number of write/erase cycles but no moving parts. NAND flash (high density) is used in SSDs; NOR flash is used for firmware.

4. Speed, Size and Cost

The **ideal memory** would be very fast, very large and very cheap - but this is impossible. **SRAM** is fast but costly, **DRAM** is cheaper but slower, **disk** is very cheap per bit but extremely slow. So computers use a **memory hierarchy** .

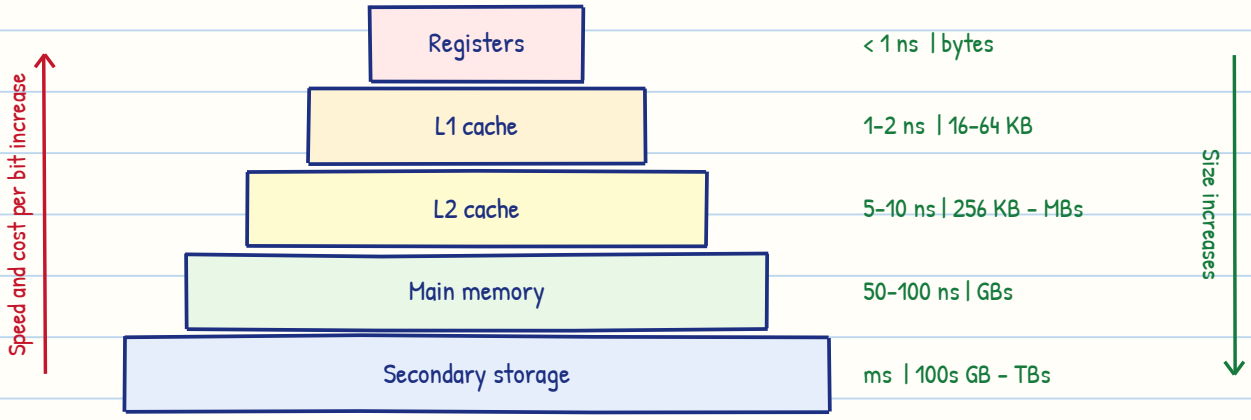


Fig 4: Memory hierarchy (typical approximate values)

- Moving **down** the hierarchy: size increases, cost per bit decreases, speed decreases.
- It works because of **locality of reference** - programs use the same or nearby data again and again, so most accesses are served by the fast upper levels.

- Result: the processor sees a memory that is **almost as fast as cache** and **as large and cheap as disk** .

5. Cache Memories

A **cache** is a small, very fast SRAM placed between the processor and main memory. It keeps copies of the **most recently used** instructions and data so the processor rarely waits for slow main memory.

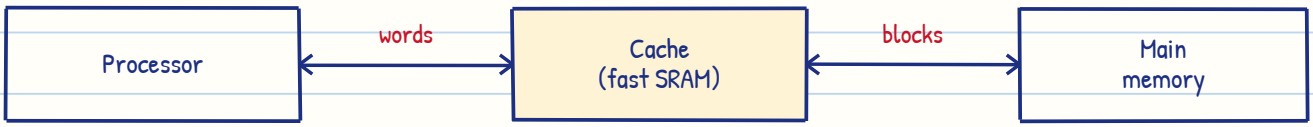


Fig 5: Cache between processor and main memory

Locality of reference

- **Temporal locality:** a recently used item will be used again soon (e.g. loop instructions).
- **Spatial locality:** items near a used address will be used soon (e.g. array elements). So a whole **block (cache line)** of consecutive words is brought in on a miss.

Basic terms

- **Hit:** requested word is in the cache. **Miss:** it is not. **Hit rate** = hits / total accesses; **miss rate** = 1 - hit rate.
- **Block (line):** group of consecutive words moved between memory and cache (e.g. 16 words).

Read and write handling

- **Read hit:** word is sent to the processor straight from cache - main memory is not involved.
- **Read miss:** the whole block is loaded from main memory. With **load-through (early restart)** the needed word is forwarded to the processor as soon as it arrives, without waiting for the full block.
- **Write hit** has two protocols:
  - **Write-through:** cache and main memory are updated together. Simple, memory is always correct, but many slow memory writes.
  - **Write-back (copy-back):** only cache is updated and the block is marked with a **dirty (modified) bit** . Memory is updated only when the block is replaced. Fewer memory writes.
- **Write miss:** write-through -> write directly to main memory. Write-back -> first load

the block into cache, then update it.

Mapping functions (where does a memory block go in the cache?)

Example used for all three methods

Main memory = 64K words (**16-bit address**). Block = 16 words -> 4096 memory blocks. Cache = 2K words = **128 blocks** .

(a) Direct mapping

- Memory block **j** can go to **only one** cache block: **j mod 128** . (Blocks 0, 128, 256 ... all compete for cache block 0.)
- Address is divided as **Tag (5) | Block (7) | Word (4)** : Block picks the cache block, Word picks the word inside it, **Tag** tells which of the 32 competing memory blocks is stored now.

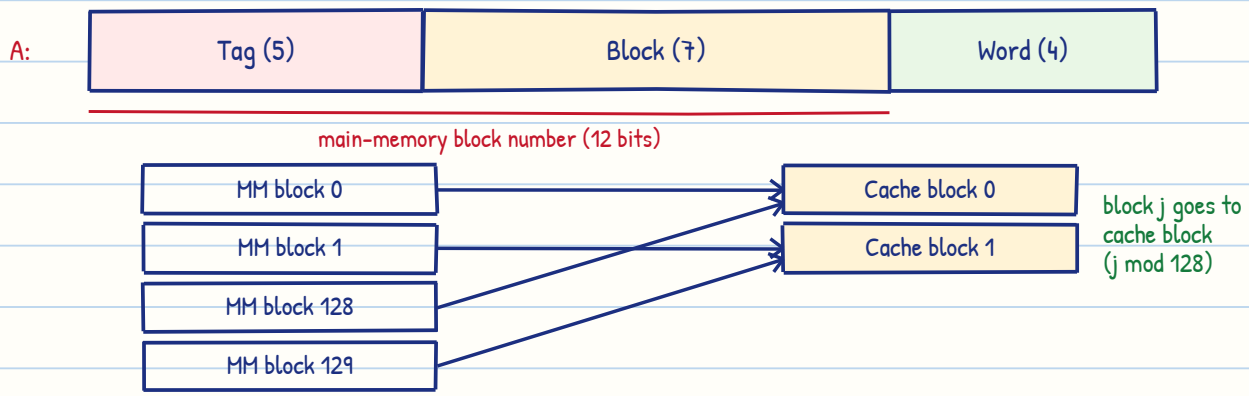


Fig 6: Direct-mapped cache

- **Hit check:** compare the tag in the address with the tag stored in that cache block (and check the **valid bit**).
- + Very simple and cheap. - Contention: two blocks that map to the same position keep replacing each other even if other cache blocks are empty.

Worked example (direct mapping)

Address 2FSC (hex) = 0010 1111 0101 1100. Word = last 4 bits = 1100 = 12. Block = next 7 bits = 1110101 = 117. Tag = first 5 bits = 00101 = 5. Check: memory block = 2FSC >> 4 = 757; 757 mod 128 = 117 and 757 / 128 = 5. So word is in cache block 117 if its stored tag = 5.

(b) Associative (fully associative) mapping

- A memory block can be placed in **any** cache block. Address = **Tag (12) | Word (4)** .

- The 12-bit tag is compared with **all 128 stored tags in parallel** . + Best use of cache, flexible. - Expensive hardware (128 comparators) and needs a replacement algorithm.

(c) Set-associative mapping

- Compromise: cache blocks are grouped into **sets** . A memory block maps to **one set** but may occupy **any block in that set** .
- Example **2-way** (2 blocks per set): 64 sets. Address = **Tag (6) | Set (6) | Word (4)** . Only the tags of the 2 blocks in the selected set are compared.
- 1-way set-associative = direct mapping; 128-way = fully associative. Real caches use 2-, 4- or 8-way.

Replacement algorithms

When the set/cache is full a block must be removed (needed only for associative and set-associative caches; direct mapping has no choice).

- **LRU** (Least Recently Used): remove the block unused for the longest time - works well due to locality; uses counters.
- **FIFO:** remove the oldest block. **Random:** remove a randomly chosen block (simple, fairly good). **LFU:** least frequently used.
- **Valid bit:** tells whether a cache block holds valid data (cleared at power-up). **Coherence:** if DMA changes main memory, the cached copy becomes stale and must be invalidated/flushed.

6. Performance Considerations

Performance depends on **hit rate (h)** and **miss penalty (M)** = extra time to bring a block from main memory.

Average access time

$t_{avg} = h \times C + (1 - h) \times M$ where C = cache access time, M = miss penalty. Example: h = 0.95, C = 1 cycle, M = 10 cycles -> $t_{avg} = 0.95 + 0.05 \times 10 = 1.45$ cycles (vs 10 cycles with no cache, i.e. about 7 times faster).

Two-level cache (L1 and L2)

$t_{avg} = h1\ C1 + (1 - h1)\ h2\ C2 + (1 - h1)(1 - h2)\ M$. Example: h1 = 0.95, h2 = 0.9, C1 = 1, C2 = 10, M = 100 -> 0.95 + 0.45 + 0.5 = 1.9 cycles.

Ways to improve performance

- **Higher hit rate:** larger cache, better mapping/replacement. Block size: bigger blocks help spatial locality but too big wastes space and raises miss penalty.
- **Interleaving:** main memory is split into several **modules** that work at the same time. With **low-order bits selecting the module** , consecutive words lie in different modules and a block can be fetched in parallel.

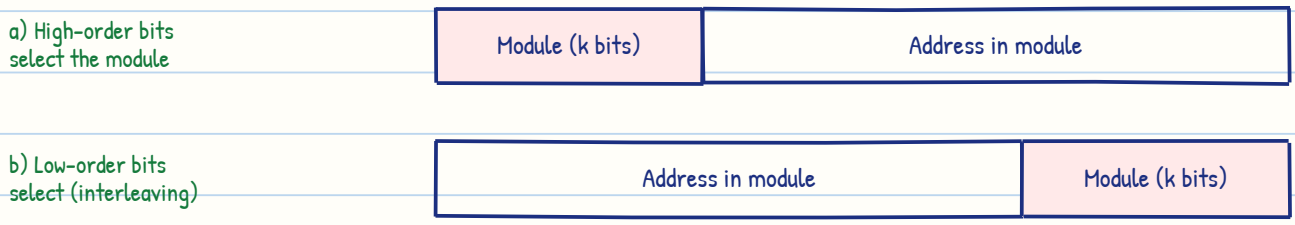


Fig 7: Address structure of memory modules

- **Example** (access = 8 cycles, transfer = 1 cycle, 4-word block): without interleaving 1 + 4 x (8 + 1) = **37 cycles** ; with 4-way interleaving 1 + 8 + 4 x 1 = **13 cycles** .
- **Write buffer:** processor puts writes in a small buffer and continues; the buffer writes to memory later.
- **Prefetching:** data is brought into the cache **before** it is requested (by special instructions or hardware).
- **Lockup-free (non-blocking) cache:** processor keeps working on other accesses while a miss is being served.
- **Separate L1 caches** for instructions and data, plus bigger **L2/L3** caches, are common.

7. Virtual Memories

Programs can be larger than main memory and many programs must share it. **Virtual memory** gives each program a large **virtual (logical) address space** ; only the needed parts are kept in main memory and the rest stays on disk. The OS and hardware move parts automatically.

- Addresses generated by the processor = **virtual addresses** . Addresses of real memory = **physical addresses** . The **MMU** (Memory Management Unit) translates virtual -> physical.
- Memory is divided into fixed-size **pages** (virtual) and **page frames** (physical), typically 2K-16K bytes.
- Virtual address = **virtual page number + offset** ; physical address = **page frame**

number + offset . The offset does not change.

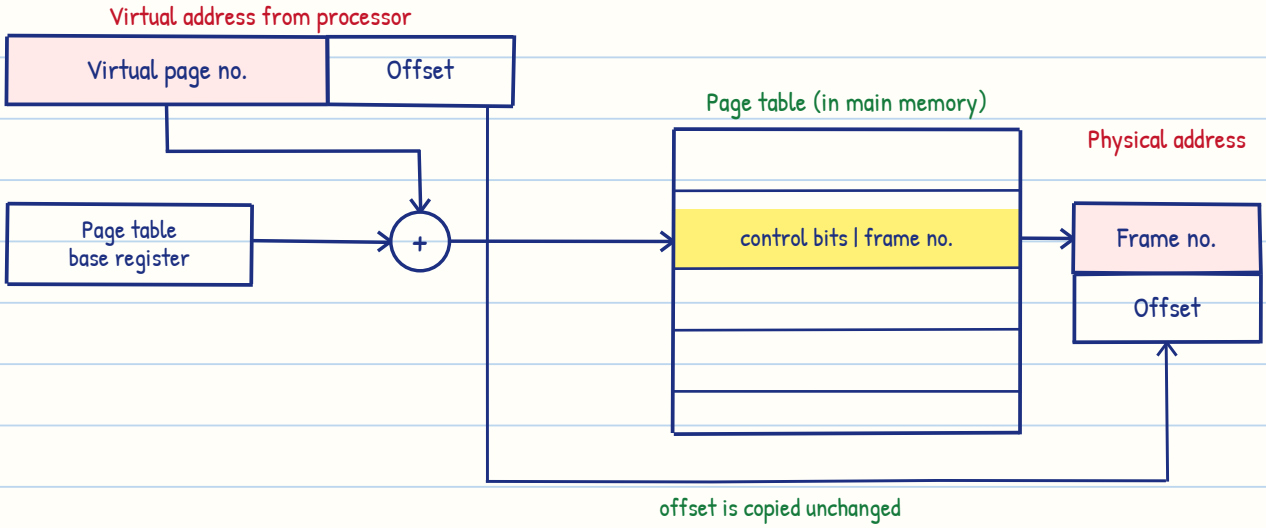


Fig 8: Virtual-to-physical address translation using the page table

Page table

- Stored in main memory. **Page table base register** + virtual page number = address of the page-table entry.
- Each entry has the **frame number** and **control bits** : **valid/present** bit, **modified (dirty)** bit, **reference** bit and **protection** (read/write) bits.

Example

32-bit virtual address with 4 KB pages: offset = 12 bits, virtual page number = 20 bits, so the page table has 2^20 = 1M entries.

TLB - Translation Lookaside Buffer

- Reading the page table in memory for every access would **double** the access time. So the MMU keeps recently used entries in a small fast **associative cache** called the **TLB** .
- **TLB hit:** frame number is available at once. **TLB miss:** entry is read from the page table and put into the TLB.

Page fault and replacement

- If the valid bit = 0 the page is not in memory -> **page fault** (an exception). The **OS** loads the page from disk (**demand paging**); meanwhile the CPU runs another program.
- If memory is full a page is replaced (LRU-like). A **dirty** page is written back to disk first; clean pages are just overwritten. **Write-back** is always used because disk writes are very slow.

Cache	Virtual memory
Speeds up main memory access	Gives illusion of a large memory
Between CPU and main memory	Between main memory and disk
Managed by hardware	Managed by OS + MMU hardware
Block size ~ 16-128 bytes; miss costs ns	Page size ~ KBs; miss costs ms

8. Memory Management Requirements

Several programs run together (**multiprogramming**), so the OS must manage memory. The memory manager (OS + hardware) must provide:

1. **Relocation:** a program can be loaded anywhere in memory (it is not known in advance). Virtual addresses are translated to physical addresses at run time.
2. **Protection:** one program must not read or write another program's (or the OS's) memory. Done with **protection bits** in the page table and **base/limit** checks.
3. **Sharing:** programs may share code or data (e.g. a library or editor) - the same physical page is mapped into several virtual spaces.
4. **Logical organisation:** programs are written as modules (code, data, stack) - supported by **segmentation** .
5. **Physical organisation:** moving information between main memory and secondary storage automatically - supported by **virtual memory** .

Processor states for protection

- **User state:** normal programs run here; cannot execute **privileged instructions** (I/O, changing MMU, etc.).
- **Supervisor (system) state:** the OS runs here with full access. A user program enters it only through an **interrupt/system call** .

Paging	Segmentation
Fixed-size pages, invisible to programmer	Variable-size logical units (code, data, stack)
Causes internal fragmentation	Causes external fragmentation
Address = page no. + offset	Address = segment no. + offset
Page table	Segment table (base and limit)

9. Secondary Storage

Secondary storage is **non-volatile, large and cheap but slow** . It holds programs and data permanently (hard disk, optical disk, tape, flash).

9.1 Magnetic hard disk

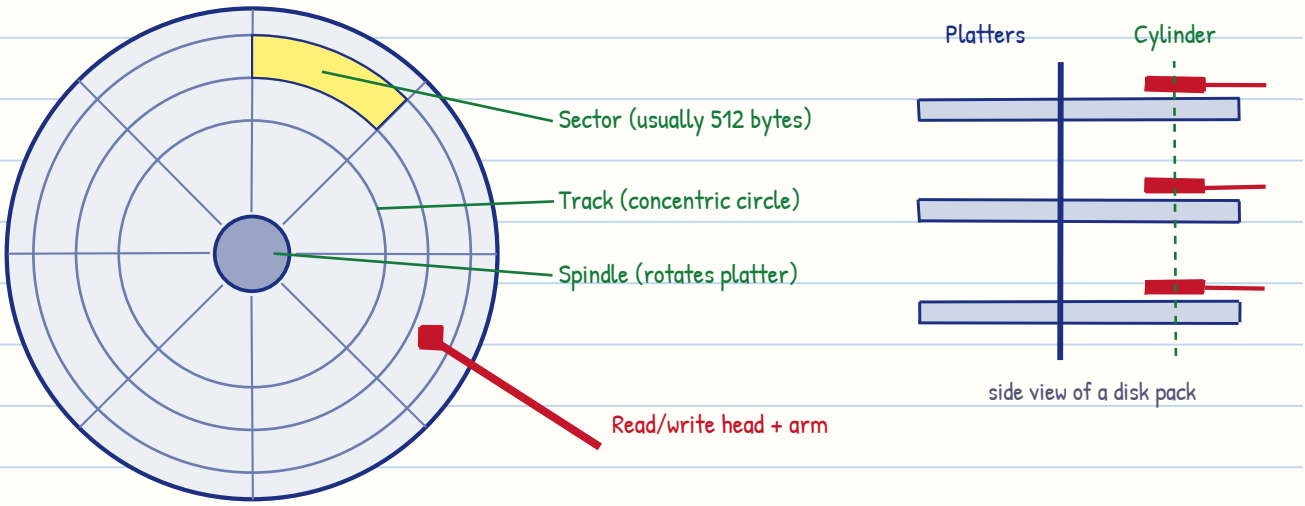


Fig 9: Disk organisation

- A disk pack has several **platters** on a **spindle** rotating at constant speed (5400 / 7200 / 15000 rpm). Each surface has a **read/write head** .
- Data is stored in **tracks** (concentric circles); each track is divided into **sectors** (usually 512 bytes). The same track on all surfaces = **cylinder** .
- A sector is addressed by **surface (head) no., track (cylinder) no. and sector no.**

Disk access time = Seek time + Rotational delay + Transfer time

Seek time: moving the head to the correct track (about 5-10 ms). **Rotational delay (latency):** waiting for the sector to arrive under the head = half a rotation on average.

Example: 7200 rpm = 120 rev/s -> one rotation = 8.33 ms -> average latency = 4.17 ms. With 6 ms seek, access is about 10 ms + a tiny transfer time.

- **Disk controller** sits between the disk and the bus: it controls seeking, reads/writes sectors, does **error checking (ECC)** and transfers data using DMA.

9.2 RAID (Redundant Array of Independent Disks)

Several disks are used together for **speed and reliability** .

- **RAID 0:** data **striped** across disks -> fast, but **no redundancy** .
- **RAID 1: mirroring** - every disk has a copy -> reliable but doubles the cost.
- **RAID 5:** striping with **distributed parity** -> survives failure of any one disk; good

balance of speed, safety and cost.

9.3 Optical, tape and flash

Device	Key points
CD-ROM / CD-R / CD-RW	Laser reads pits and lands; ~650-700 MB; read-only / write-once / rewritable
DVD	Shorter-wavelength laser, 4.7 GB per layer (more with two layers/sides)
Blu-ray	Blue-violet laser, about 25 GB per layer
Magnetic tape	Sequential access only; very cheap; used for backup and archives
Flash / SSD	No moving parts, fast random access, shock-proof; costlier per GB than disk

Quick Revision and Important Questions

- Memory hierarchy: registers > cache > main memory > disk (size up, speed down, cost/bit down).
- SRAM = latch, no refresh, fast. DRAM = capacitor, refresh, cheap. SDRAM/DDR = clocked, burst, double rate.
- ROM family: ROM, PROM, EPROM, EEPROM, Flash. Mapping: direct, associative, set-associative.
- $t_{avg} = hC + (1 - h)M$. Virtual address = page no. + offset; TLB caches page-table entries.

Likely exam questions

1. Explain the organisation of a memory chip with a neat diagram.
2. Differentiate SRAM and DRAM. Explain the working of a DRAM cell and refresh.
3. Explain the types of ROM.
4. Explain the memory hierarchy and the concept of locality of reference.
5. Explain direct, associative and set-associative mapping with address formats.
6. Explain write-through and write-back; replacement algorithms.
7. How does interleaving improve memory performance? Derive average access time.
8. Explain virtual memory, address translation, page table and TLB.
9. Explain the memory management requirements.
10. Explain disk organisation and access time; write short notes on RAID and optical disks.